PROJECT DELIVERY & QUALITY ASSURANCE REPORT

Premium Pet Care Booking Platform

High-fidelity, multi-page booking web app (brand: Willow) — caregiver discovery, scheduling and an in-app wallet, delivered as a zero-dependency web app.

PROJECT	VERSION
Premium Pet Care Booking	**1.0**
PREPARED	STACK
07 Nov 2026	**HTML / CSS / Vanilla JS**

01 Executive Summary

A high-fidelity, enterprise-grade frontend prototype for a premium pet-care services marketplace — delivered as a zero-dependency, multi-page web app in pure native HTML, CSS and Vanilla JavaScript.

The product guides a pet owner through a complete booking journey — service selection, caregiver discovery with live availability, pet-profile capture, scheduling and confirmation — and pairs it with an in-app wallet and transaction ledger. A light / dark theme is built in and persisted across sessions. As with the rest of the suite, no framework, build step or network request is required to run it.

All data access is funnelled through the same custom **App-Runtime** (`app-runtime.js`, v1.0.0): a swappable data adapter, skeleton loading, and a recoverable error + retry surface. The prototype runs on mock data today and against a live API tomorrow with a single flag flip.

02 Core Features

Four capabilities separate this build from a template: each is implemented as deterministic logic, not decorative markup.

- **Debounced caregiver discovery & live availability search** — keystrokes are coalesced before a single async-flavoured filter pass; the directory never stutters under rapid input.
- **Multi-step, cross-page booking state machine** — service → caregiver → pet → date → confirm, with guarded back / forward navigation and resilient state restoration.
- **Dynamic pricing engine** — line-item totals (service, duration, pet count) recompute on every change and always reconcile in the order summary.
- **Self-clearing timers & wallet ledger** — session timers are torn down on view teardown; the running balance ledger updates without leaking intervals.

03 UI / UX Design Specification

The interface is governed by a single token set declared as CSS custom properties, with a dual light / dark theme swapped through the same variables — so the palette never drifts.

TOKEN	VALUE (LIGHT)	ROLE
`--bg`	#FBF8F2 · Cream	Page background (dark: #151B17)
`--surface`	#FFFDF9 · Paper	Cards, sheets (dark: #1C241E)
`--ink`	#2C3A31 · Sage Ink	Headings & body (dark: #EDF3EC)
`--muted`	#89958A · Muted Sage	Captions, metadata
`--accent`	#4E7159 · Deep Sage	Primary buttons, active states
`--accent-2`	#C6A24C · Gold	CTAs, emphasis
`--terra`	#C87C5C · Terracotta	Secondary highlight
`--line`	#E9E5DA · Stone Line	Borders, hairline dividers

SCALE / SPEC	VALUE	NOTE
Type	40/700 · 28/700 · 18/700 · 16/400 · 13/500 · 12/600	Display → Mono, line-height 1.6
Radius	8 · 12 · 16 · 24	Inputs / cards / hero blocks
Theme	Light + Dark via CSS variables	Persisted to localStorage

04 Architecture & Code Standards

`app-runtime.js` is the one piece of shared infrastructure. It is framework-free, fully commented, and exposes a small, intentional API.

```
// app-runtime.js — v1.0.0  (zero dependencies, pure Vanilla JS)
AppRuntime = {
  apiRequest(url, opts)      // Axios-shaped fetch: 8s timeout, Bearer token, typed ApiError
  dataAdapter               // the single seam:  mock <-> remote  (setMode)
    .request(endpoint, opts) // UI never touches fetch directly
    .setMode('mock' | 'remote')
  showLoading(host, label)  // skeleton state
  hideLoading(host)
  showError(host, msg, retry) // recoverable error + Retry button
  mount(host, boot, opts)    // lifecycle: skeleton -> request -> render / retry
  debounce(fn, ms)           // search coalescing
  throttle(fn, ms)           // resize broadcast guard
}
```

Pure Vanilla JS — no React, no Vue, no jQuery. Mock data lives at the top of each file as the single source of truth and is rendered through DOM templates with event delegation. Because the data layer is isolated behind `dataAdapter`, a backend engineer only replaces one module to go live; the entire UI, loading and error behaviour stays intact.

- Token-driven CSS — colours, spacing and radii are `:root` variables; theme swap is a single variable override.
- Mobile-first — base styles target the phone; widen with `@media (min-width:…)`.
- Accessible — labelled inputs, visible focus, `prefers-reduced-motion` guards on heavy animation.
- Commented business logic — pricing math, state transitions and timers carry clear explanations.

05 QA / Test Report Matrix

Every assertion below is reproducible in a browser with no backend attached. All statuses are **PASS**.

CATEGORY	TEST CASE	EXPECTED RESULT	STATUS
Runtime	Simulate offline / HTTP 500 via `dataAdapter.setMode('remote')`	Skeleton renders, ApiError caught, Retry button shown, UI recovers on retry	PASS
Search	Fire 30 keystrokes in 1 s into caregiver search	Debounce collapses to one filter pass; no flicker, no duplicate nodes	PASS
Booking	Drive the multi-page state machine through every legal + illegal transition	Only legal transitions apply; illegal ones rejected without throwing	PASS
Pricing	Change service / duration / pet count repeatedly	Totals recompute deterministically; order summary reconciles	PASS
Timers	Mount / unmount the wallet view 50× in a loop	Zero leaked intervals; heap remains flat	PASS
Theme	Toggle dark mode, reload the page	Theme persists via localStorage; no flash of wrong theme	PASS
Responsive	320 / 375 / 768 / 1280 px viewports	Layout intact; no horizontal overflow	PASS
Browser	Chrome / Safari / Firefox / Edge (latest)	Identical render; no vendor prefixes required	PASS
A11y	Labels, focus order, reduced-motion	Baseline present; fully keyboard reachable	PASS

06 Delivery Checklist

Sign-off summary for hand-off.

- [x] **Functional completeness**
  All in-scope flows are clickable and produce correct state changes.
- [x] **Visual quality**
  Token-driven design system with dual light / dark theme.
- [x] **Responsive compliance**
  Mobile-first; verified at 320 / 375 / 768 / 1280 px.
- [x] **Performance**
  Zero external requests; opens instantly, fully offline.
- [x] **Content integrity**
  Fictional brands only; no real company names used.
- [x] **Code standards**
  Vanilla JS, no dependencies, commented business logic.

NEXT STEPS (ANTI-PITFALL CLAUSE)

Front-end is delivered production-ready. The following are **explicitly out of scope** for this phase and belong to the backend / integration team — do not treat the prototype's mock success states as live transactions:

1. Wire real REST / GraphQL endpoints into `dataAdapter` and flip mode to `'remote'`.
2. Integrate a payment gateway (Stripe / PayPal) for paid bookings and wallet top-ups.
3. Replace mock auth with real session / JWT handling and caregiver verification.
4. Swap CSS-gradient and emoji placeholders for licensed brand photography.